

LABORATORY WORK 5

PostgreSQL Constraints

Database Management Systems

Task 1 — Minimum passenger age

Explanation: A CHECK constraint was added to the passengers table. It allows only passengers who are at least 10 years old.

SQL Query

```
ALTER TABLE passengers
ADD CONSTRAINT ck_passenger_age
CHECK (date_of_birth <= CURRENT_DATE - INTERVAL '10 years');
```

Result: The constraint/query was executed successfully.

Screenshot — Task 1

The screenshot displays a PostgreSQL database management interface. On the left, the 'Object Explorer' shows a tree view of the database schema, including tables like 'booking', 'passengers', and 'airline'. The 'passengers' table is selected, showing its columns: 'passenger_id' (integer), 'first_name' (character varying(50)), 'last_name' (character varying(50)), 'date_of_birth' (date), 'gender' (character varying(50)), 'country_of_citizenship' (character varying(50)), 'country_of_residence' (character varying(50)), 'passport_number' (character varying(50)), 'created_at' (date), and 'updated_at' (date). The 'Constraints' section for 'passengers' is expanded, showing a new constraint 'ck_passenger_age' of type 'CHECK'.

The main window shows the 'Query' tab with the following SQL script:

```
--1
ALTER TABLE passengers
ADD CONSTRAINT ck_passenger_age
CHECK (date_of_birth <= CURRENT_DATE - INTERVAL '10 years');
--2
ALTER TABLE booking
ADD CONSTRAINT ck_booking_price
CHECK (price BETWEEN 0 AND 50000);
--3
ALTER TABLE baggage
ADD CONSTRAINT ck_baggage_weight
CHECK (weight_in_kg BETWEEN 1 AND 23);
--4
ALTER TABLE airport
ADD CONSTRAINT ck_airport_name_length
CHECK (LENGTH(airport_name) >= 10);
--5
ALTER TABLE passengers
ADD CONSTRAINT uq_passport_number
UNIQUE (passport_number);
ALTER TABLE airline
```

The 'Data Output' tab shows the results of the query execution, displaying a single row with the following data:

booking_id	created_at
502	2026-10-07

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.051'.

Task 2 — Booking price

Explanation: A CHECK constraint was added to the booking table. The price must be between 0 and 50,000.

SQL Query

```
ALTER TABLE booking
ADD CONSTRAINT ck_booking_price
CHECK (price BETWEEN 0 AND 50000);
```

Result: The constraint/query was executed successfully.

Screenshot — Task 2

The screenshot displays a PostgreSQL IDE interface with the following components:

- Object Explorer (Left):** Shows the database schema with tables: booking, baggage, airport, passengers, and airline. The 'passengers' table is currently selected, showing its columns: passenger_id, first_name, last_name, date_of_birth, gender, country_of_citizenship, country_of_residence, passport_number, and created_at.
- Query Editor (Center):** Contains a series of SQL queries executed sequentially:
 - Query 5: `ALTER TABLE booking`
 - Query 6: `--2`
 - Query 7: `ALTER TABLE booking`
 - Query 8: `ADD CONSTRAINT ck_booking_price`
 - Query 9: `CHECK (price BETWEEN 0 AND 50000);`
 - Query 10: `--3`
 - Query 11: `ALTER TABLE baggage`
 - Query 12: `ADD CONSTRAINT ck_baggage_weight`
 - Query 13: `CHECK (weight_in_kg BETWEEN 1 and 23);`
 - Query 14: `--4`
 - Query 15: `ALTER TABLE airport`
 - Query 16: `ADD CONSTRAINT ck_airport_name_length`
 - Query 17: `CHECK (LENGTH(airport_name) >= 10 );`
 - Query 18: `--5`
 - Query 19: `ALTER TABLE passengers`
 - Query 20: `ADD CONSTRAINT uq_passport_number`
 - Query 21: `UNIQUE (passport_number);`
 - Query 22: `ALTER TABLE airline`
 - Query 23: `ADD CONSTRAINT uq_airline_code`
 - Query 24: `UNIQUE (airline_code);`
 - Query 25: `ALTER TABLE airport`
 - Query 26: `ADD CONSTRAINT uq_airport_name`
- Data Output (Bottom):** Shows the result of the first query, displaying a single row from the 'booking' table:

booking_id	created_at
1	2026-10-07
- Status Bar (Bottom):** Indicates 'Total rows: 1' and 'Query complete 00:00:00.051'.

Task 3 — Baggage weight

Explanation: A CHECK constraint was added to baggage. The baggage weight must be from 1 to 23 kg.

SQL Query

```
ALTER TABLE baggage
ADD CONSTRAINT ck_baggage_weight
CHECK (weight_in_kg BETWEEN 1 AND 23);
```

Result: The constraint/query was executed successfully.

Screenshot — Task 3

The screenshot displays a PostgreSQL IDE interface. On the left, the 'Object Explorer' shows a database schema with tables like 'booking', 'passengers', and 'flights'. The 'passengers' table is selected, showing its columns: 'passenger_id' (integer), 'first_name' (character varying(50)), 'last_name' (character varying(50)), 'date_of_birth' (date), 'gender' (character varying(50)), 'country_of_citizenship' (character varying(50)), 'country_of_residence' (character varying(50)), 'passport_number' (character varying(50)), and 'created_at' (date). The 'Query' tab is active, showing a multi-line SQL script. The script includes comments and SQL commands to add constraints to several tables: 'baggage', 'airport', 'passengers', 'airline', and 'passengers' again. The specific query being executed is:
--3
ALTER TABLE baggage
ADD CONSTRAINT ck_baggage_weight
CHECK (weight_in_kg BETWEEN 1 AND 23);
The 'Data Output' tab shows the result of the query, which is a single row with the following data:

booking_id	created_at
1	2026-10-07

 The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.051'.

Task 4 — Airport name length

Explanation: A CHECK constraint was added to airport. The airport name must contain at least 10 characters.

SQL Query

```
ALTER TABLE airport
ADD CONSTRAINT ck_airport_name_length
CHECK (LENGTH(airport_name) >= 10);
```

Result: The constraint/query was executed successfully.

Screenshot — Task 4

The screenshot displays a PostgreSQL IDE interface. On the left, the 'Object Explorer' shows a database schema with tables like 'booking', 'passengers', and 'flights'. The 'passengers' table is selected, showing its columns: 'passenger_id' (integer), 'first_name' (character varying(50)), 'last_name' (character varying(50)), 'date_of_birth' (date), 'gender' (character varying(50)), 'country_of_citizenship' (character varying(50)), 'country_of_residence' (character varying(50)), 'passport_number' (character varying(50)), 'created_at' (date), and 'update_at' (date). The main window shows a SQL query editor with the following code:

```
--4
ALTER TABLE airport
ADD CONSTRAINT ck_airport_name_length
CHECK (LENGTH(airport_name) >= 10 );
--5
ALTER TABLE passengers
ADD CONSTRAINT uq_passport_number
UNIQUE (passport_number);
--6
ALTER TABLE airline
ADD CONSTRAINT uq_airline_code
UNIQUE (airline_code);
--7
ALTER TABLE airport
ADD CONSTRAINT uq_airport_name
UNIQUE (airport_name);
--8
ALTER TABLE passengers
ADD CONSTRAINT ck_passenger_gender_age
CHECK (
  (gender = 'Male' AND date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
  OR
  (gender = 'Female' AND date_of_birth <= CURRENT_DATE - INTERVAL '19 years')
);
```

Below the query editor, the 'Data Output' tab shows the results of the query. It displays a table with two columns: 'booking_id' (PK) and 'created_at'. The data shows one row with 'booking_id' 1 and 'created_at' 2026-10-07.

booking_id (PK)	created_at
1	2026-10-07

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.051'.

Task 5 — UNIQUE constraints

Explanation: UNIQUE constraints were added to columns that should contain unique values: passport number, airline code and airport name. This prevents duplicate values in these columns.

SQL Query

```
ALTER TABLE passengers
ADD CONSTRAINT uq_passport_number
UNIQUE (passport_number);
```

```
ALTER TABLE airline
ADD CONSTRAINT uq_airline_code
UNIQUE (airline_code);
```

```
ALTER TABLE airport
ADD CONSTRAINT uq_airport_name
UNIQUE (airport_name);
```

Result: The constraint/query was executed successfully.

Screenshot — Task 5

The screenshot displays a PostgreSQL IDE interface. On the left, the 'Object Explorer' shows a tree view of database objects, including tables like 'booking', 'flights', and 'passengers'. The 'passengers' table is selected, showing its columns: 'passenger_id' (integer), 'first_name' (character varying(50)), 'last_name' (character varying(50)), 'date_of_birth' (date), 'gender' (character varying(50)), 'country_of_citizenship' (character varying(50)), 'country_of_residence' (character varying(50)), 'passport_number' (character varying(50)), 'created_at' (date), and 'update_at' (date). The main query editor shows the following SQL queries:

```
--5
ALTER TABLE passengers
ADD CONSTRAINT uq_passport_number
UNIQUE (passport_number);

--6
ALTER TABLE airline
ADD CONSTRAINT uq_airline_code
UNIQUE (airline_code);

--7
ALTER TABLE airport
ADD CONSTRAINT uq_airport_name
UNIQUE (airport_name);

--6
ALTER TABLE passengers
ADD CONSTRAINT ck_passenger_gender_age
CHECK (
  (gender = 'Male' AND date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
  OR
  (gender = 'Female' AND date_of_birth <= CURRENT_DATE - INTERVAL '19 years')
);

--7
ALTER TABLE passengers
ADD CONSTRAINT ck_passenger_citizenship_age
CHECK (
```

Below the query editor, the 'Data Output' tab shows a table with the following data:

booking_id	created_at
1	2026-10-07

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.051'.

Task 6 — Gender and age

Explanation: A CHECK constraint was added so that male passengers must be at least 18 years old and female passengers must be at least 19 years old.

SQL Query

```
ALTER TABLE passengers
ADD CONSTRAINT ck_passenger_gender_age
CHECK (
    (gender = 'Male' AND date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
    OR
    (gender = 'Female' AND date_of_birth <= CURRENT_DATE - INTERVAL '19 years')
);
```

Result: The constraint/query was executed successfully.

Screenshot — Task 6

The screenshot displays a PostgreSQL IDE interface. On the left, the 'Object Explorer' shows the database structure, including the 'passengers' table with its columns and constraints. The main editor window shows the SQL query being executed, which adds a CHECK constraint named 'ck_passenger_gender_age' to the 'passengers' table. The query is as follows:

```
--6
ALTER TABLE passengers
ADD CONSTRAINT ck_passenger_gender_age
CHECK (
    (gender = 'Male' AND date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
    OR
    (gender = 'Female' AND date_of_birth <= CURRENT_DATE - INTERVAL '19 years')
);
```

Below the query editor, the 'Data Output' tab shows the result of the query, which is a single row indicating the successful execution of the command. The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.051'.

booking_id	created_at
1	2026-10-07

Task 7 — Citizenship and age

Explanation: The age requirement depends on citizenship. Kazakhstan citizens must be at least 18, France citizens at least 17, and citizens of other countries at least 19.

SQL Query

```
ALTER TABLE passengers
ADD CONSTRAINT ck_passenger_citizenship_age
CHECK (
    (country_of_citizenship = 'Kazakhstan'
    AND date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
    OR
    (country_of_citizenship = 'France'
    AND date_of_birth <= CURRENT_DATE - INTERVAL '17 years')
    OR
    (country_of_citizenship NOT IN ('Kazakhstan', 'France')
    AND date_of_birth <= CURRENT_DATE - INTERVAL '19 years')
);
```

Result: The constraint/query was executed successfully.

Screenshot — Task 7

The screenshot displays a PostgreSQL IDE interface. On the left, the Object Explorer shows the database structure, including tables like booking, passengers, and flights. The main query editor window shows the following SQL script:

```
--7
ALTER TABLE passengers
ADD CONSTRAINT ck_passenger_citizenship_age
CHECK (
    (country_of_citizenship = 'Kazakhstan'
    AND date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
    OR
    (country_of_citizenship = 'France'
    AND date_of_birth <= CURRENT_DATE - INTERVAL '17 years')
    OR
    (country_of_citizenship NOT IN ('Kazakhstan', 'France')
    AND date_of_birth <= CURRENT_DATE - INTERVAL '19 years')
);

--8
ALTER TABLE booking
ADD COLUMN ticket_discount NUMERIC;

UPDATE booking
SET ticket_discount = 5
WHERE created_at > '2024-01-01';

UPDATE booking
SET ticket_discount = 10
WHERE created_at < '2024-01-01';
```

Below the query editor, the Data Output tab shows the results of the query, displaying a single row from the booking table:

booking_id	created_at
1	2026-10-07

The status bar at the bottom indicates "Total rows: 1" and "Query complete 00:00:00.051".

Task 8 — Ticket discount

Explanation: A ticket_discount column was added to booking. Bookings created after 2024-01-01 receive a 5% discount, while bookings created before 2024-01-01 receive a 10% discount. A CHECK constraint enforces this rule.

SQL Query

```
ALTER TABLE booking
ADD COLUMN ticket_discount NUMERIC;

UPDATE booking
SET ticket_discount = 5
WHERE created_at > '2024-01-01';

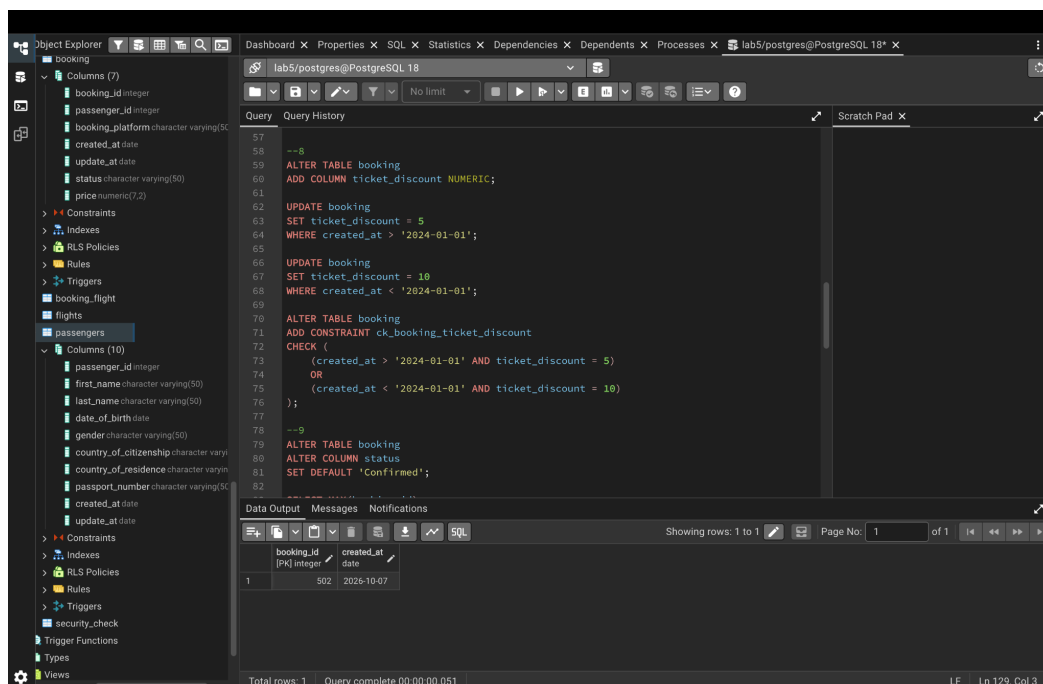
UPDATE booking
SET ticket_discount = 10
WHERE created_at < '2024-01-01';

UPDATE booking
SET created_at = '2023-12-31'
WHERE booking_id = 421;

ALTER TABLE booking
ADD CONSTRAINT ck_booking_ticket_discount
CHECK (
    (created_at > '2024-01-01' AND ticket_discount = 5)
    OR
    (created_at < '2024-01-01' AND ticket_discount = 10)
);
```

Result: The constraint/query was executed successfully.

Screenshot — Task 8



Task 9 — Default booking status

Explanation: The status column was given the default value 'Confirmed'. A new booking was inserted without specifying status, and PostgreSQL automatically assigned 'Confirmed'.

SQL Query

```
ALTER TABLE booking
ALTER COLUMN status
SET DEFAULT 'Confirmed';

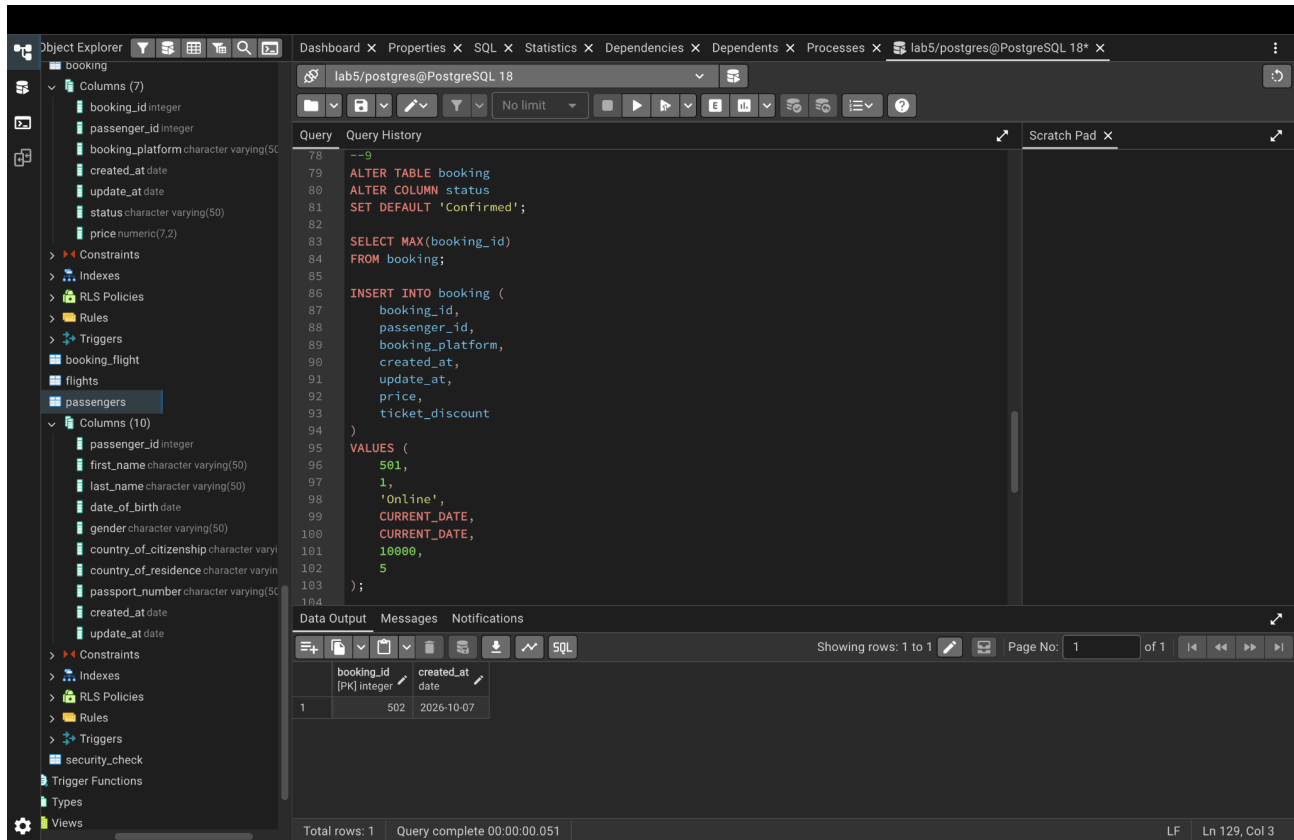
SELECT MAX(booking_id)
FROM booking;

INSERT INTO booking (
    booking_id,
    passenger_id,
    booking_platform,
    created_at,
    update_at,
    price,
    ticket_discount
)
VALUES (
    501,
    1,
    'Online',
    CURRENT_DATE,
    CURRENT_DATE,
    10000,
    5
);

SELECT booking_id, status
FROM booking
WHERE booking_id = 501;
```

Result: The query was executed successfully and the result was verified in pgAdmin.

Screenshot — Task 9



Task 10 — Current date by default

Explanation: The created_at column was given CURRENT_DATE as its default value. A new booking was inserted without created_at, and PostgreSQL automatically stored the current date.

SQL Query

```
ALTER TABLE booking
ALTER COLUMN created_at
SET DEFAULT CURRENT_DATE;
```

```
INSERT INTO booking (
    booking_id,
    passenger_id,
    booking_platform,
    update_at,
    price,
    ticket_discount
)
VALUES (
    502,
    1,
    'Online',
    CURRENT_DATE,
    10000,
    5
);
```

```
SELECT booking_id, created_at
FROM booking
WHERE booking_id = 502;
```

Result: The query was executed successfully and the result was verified in pgAdmin.

Screenshot — Task 10

The screenshot displays the pgAdmin 4 interface with the following components:

- Object Explorer (Left):** Shows the database structure for 'lab5/postgres@PostgreSQL 18'. The 'booking' table is selected, showing columns: booking_id (integer), passenger_id (integer), booking_platform (character varying(50)), created_at (date), update_at (date), status (character varying(50)), and price (numeric(7,2)).
- Query Editor (Center):** Contains the following SQL script:

```
--10
ALTER TABLE booking
ALTER COLUMN created_at
SET DEFAULT CURRENT_DATE;

INSERT INTO booking (
    booking_id,
    passenger_id,
    booking_platform,
    update_at,
    price,
    ticket_discount
)
VALUES (
    502,
    1,
    'Online',
    CURRENT_DATE,
    10000,
    5
);

SELECT booking_id, created_at
FROM booking
WHERE booking_id = 502;
```
- Data Output (Bottom):** Shows the results of the final query. It displays a table with two columns: 'booking_id' (integer) and 'created_at' (date). The results are as follows:

booking_id	created_at
502	2026-10-07
- Query History (Top Right):** Shows the executed query.
- Messages (Bottom Right):** Shows the status of the query execution.

Conclusion

During Laboratory Work 5, PostgreSQL constraints were added and tested on the airline database. CHECK constraints were used to control age, price, baggage weight, airport name length and discount rules. UNIQUE constraints were used to prevent duplicate passport numbers, airline codes and airport names. DEFAULT values were also configured for booking status and creation date. The results were checked in pgAdmin after executing the SQL statements.